

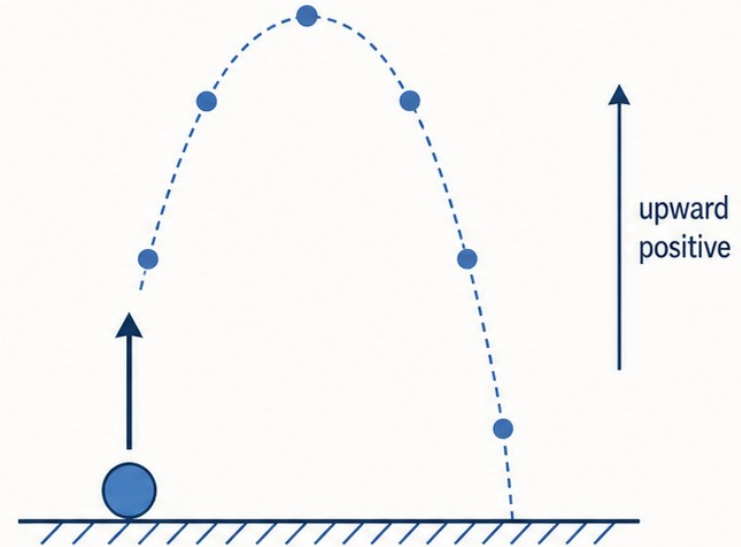
From one calculation to a traceable loop

A familiar equation can be evaluated one time at a time.

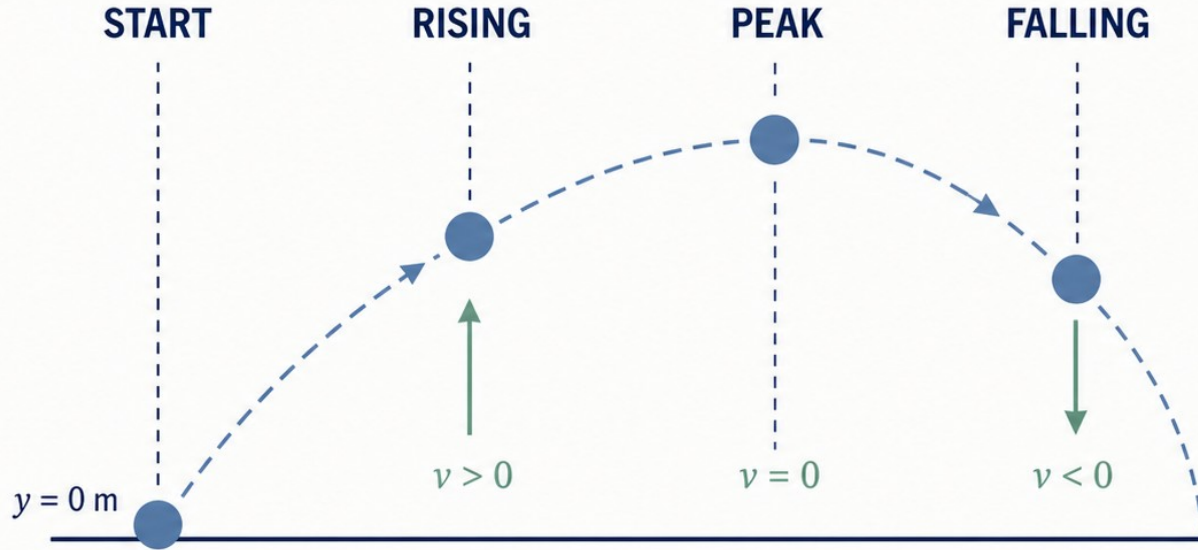
An algorithm makes the order visible.

A loop makes repetition explicit.

Evidence still needs checking and physical interpretation.



Predict the motion before asking MATLAB to repeat anything



Choose upward as positive.



The ball starts at $y = 0 \text{ m}$.



It rises to one maximum, then falls.



The prediction is our first debugging reference.



What should the output look like?

Fix the model, then ask what repeats

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

$$y_0 = 0 \text{ m}$$

$$v_0 = 20 \text{ m s}^{-1}$$

$$g = 9.81 \text{ m s}^{-2}$$



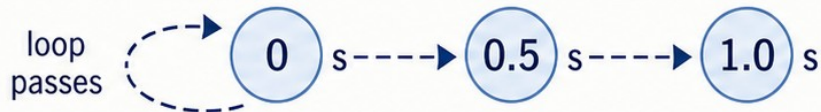
current time: $t_s = 1.0 \text{ s}$



one evaluation
→ one position

$$y(1.0 \text{ s}) = 15.0950 \text{ m}$$

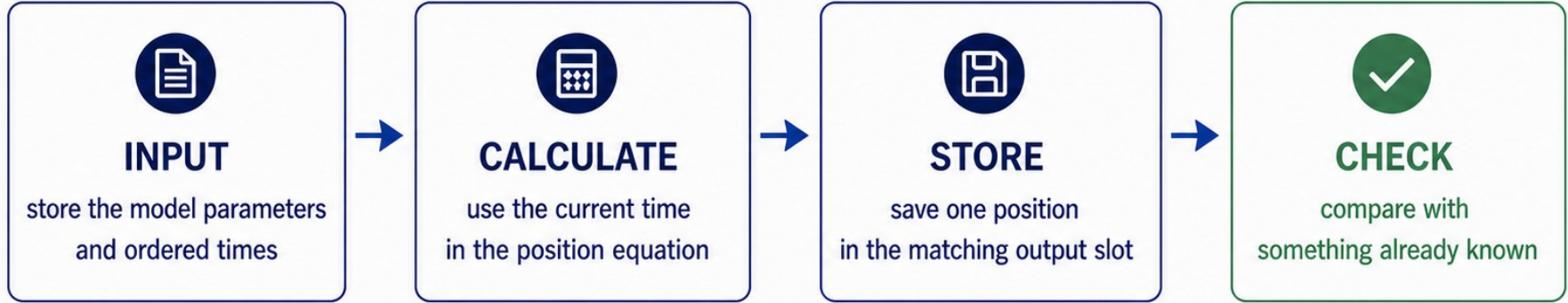
The model stays fixed; current_time_s changes.



Choose ordered times: $t_s = 0:0.5:4 \text{ s}$

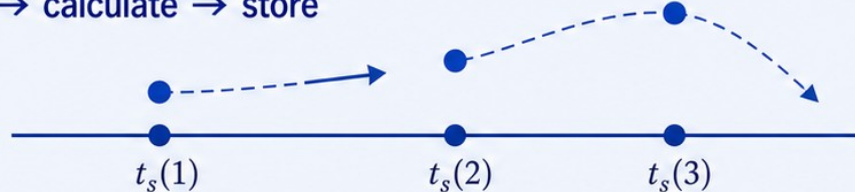
One calculation is four small jobs

Before MATLAB syntax, make the order of reasoning visible.



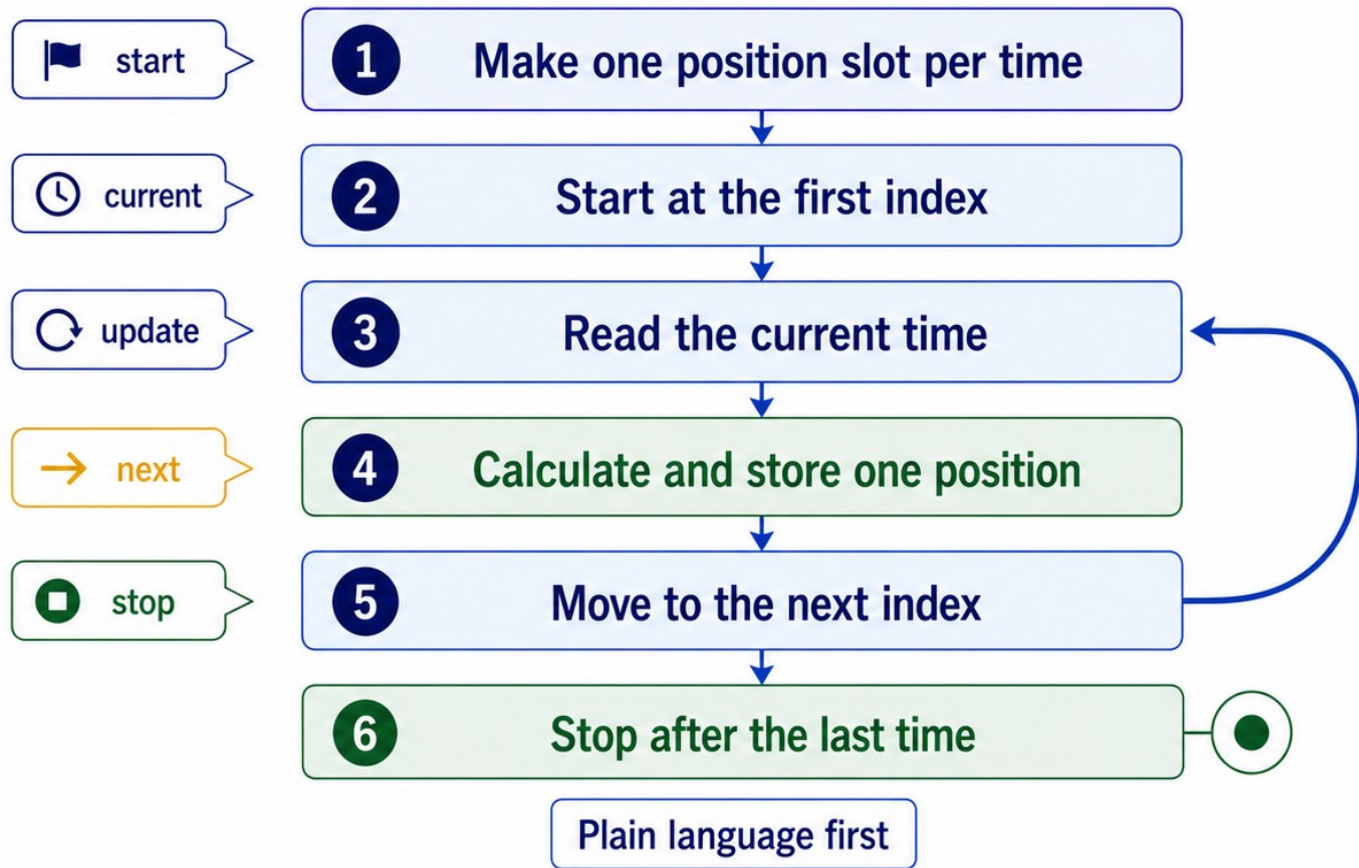
One pass = one time $\rightarrow$ one position

One pass: select $\rightarrow$ calculate $\rightarrow$ store



same recipe • new sample

Turn the jobs into a repeatable recipe



The index is a storage address, not a time

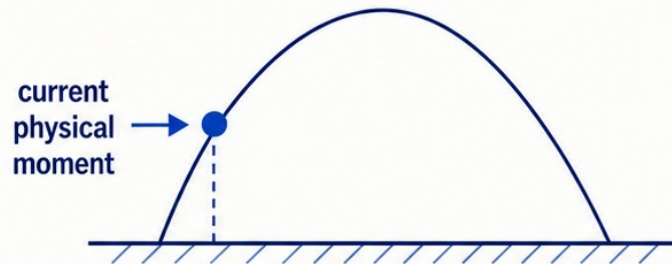
time array

	1	2	3	
$t_s =$	0	0.5	1.0	...

position array

	1	2	3	
$y_m =$	0	8.77375	15.0950	...

sample_index = 2



$t_s(\text{sample_index}) = 0.5 \text{ s}$

$y_m(\text{sample_index}) = 8.77375 \text{ m}$

1

MATLAB indexing starts at 1.



$t_s(\text{sample_index})$ selects the current physical time.



$y_m(\text{sample_index})$ selects the matching output slot.



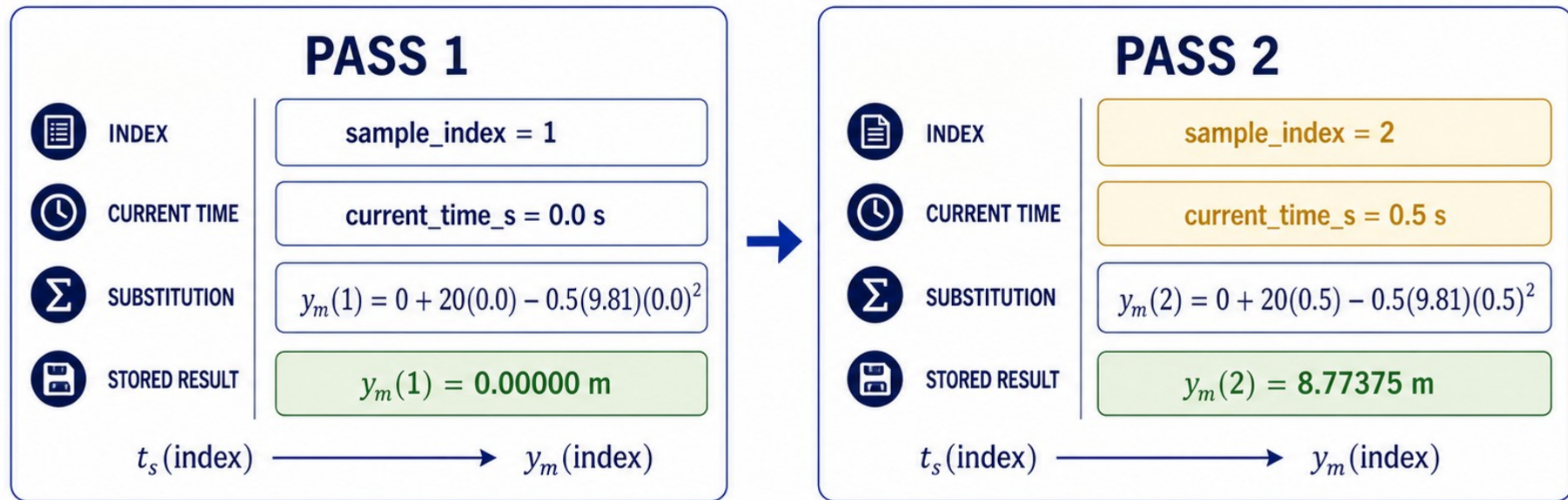
The same index keeps input and output aligned.



Same index → matching input and output

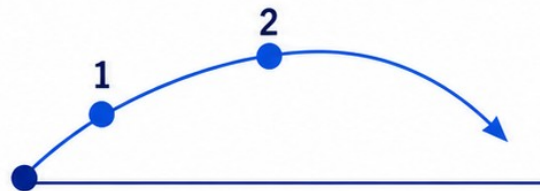
Trace two passes before trusting the whole loop

Trace symbols first; interpret the number physically second.



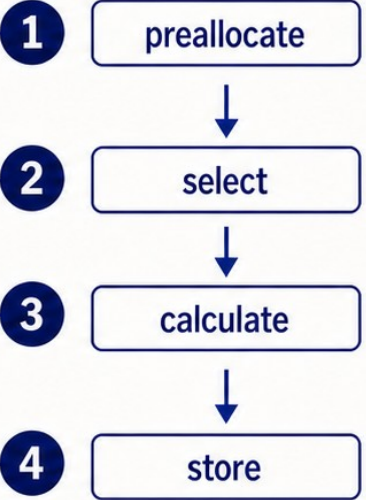
index 1 → output slot 1

index 2 → output slot 2



Write the MATLAB loop that matches the recipe

```
y_m = zeros(size(t_s));  
  
for sample_index = 1:n_samples  
    current_time_s = t_s(sample_index);  
  
    y_m(sample_index) = y0_m + v0_mps*current_time_s ...  
        - 0.5*g_mps2*current_time_s^2;  
  
end
```



One pass handles
one scalar time

Complete one line, then say what it does

```
y_m = zeros(size(t_s));  
for sample_index = 1:n_samples  
    current_time_s = t_s(sample_index);  
    y_m(sample_index) = _____;  
end
```

```
y0_m + v0_mps*current_time_s - 0.5*g_mps2*current_time_s^2
```



current time



equation



matching slot



The missing line is the position update.



Use the current time, not the whole time array.



Store into y_m(sample_index).

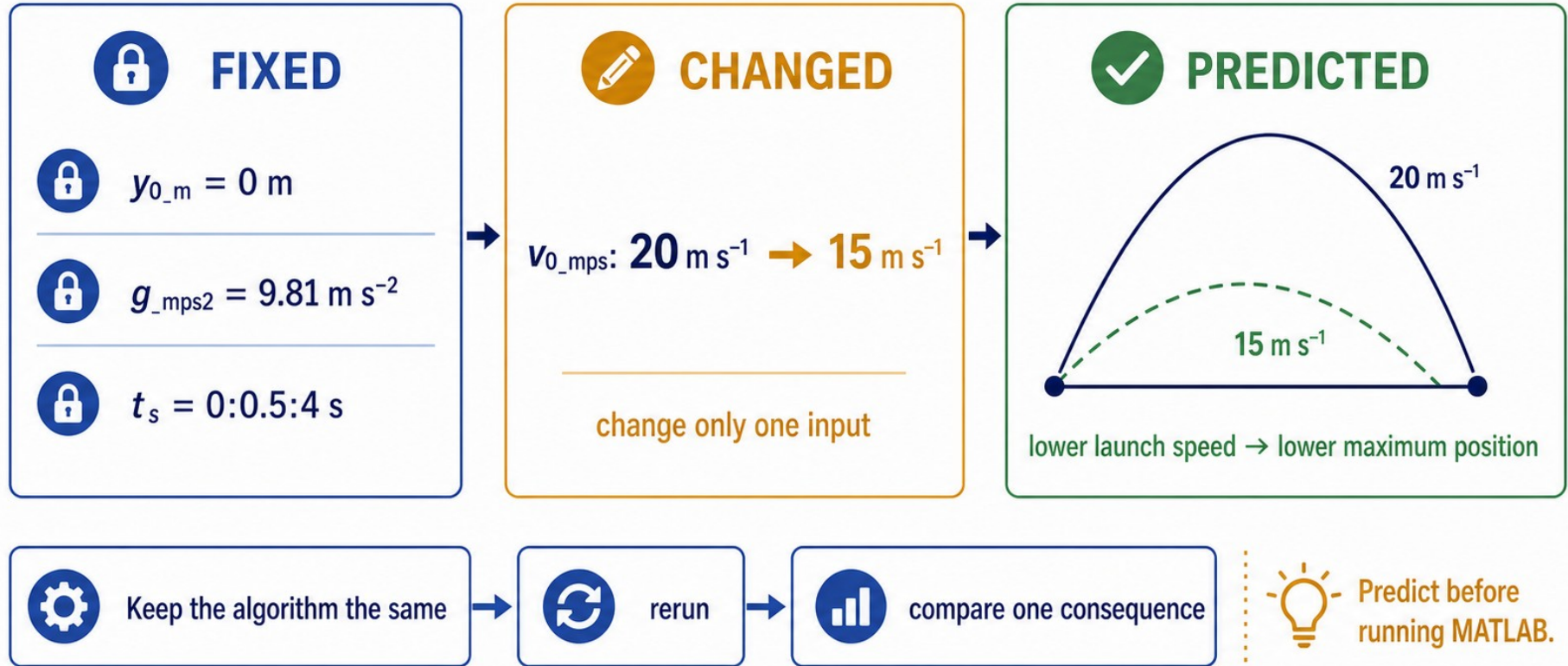


FIRST-VALUE CHECK

$y_m(1) = 0 \text{ m}$

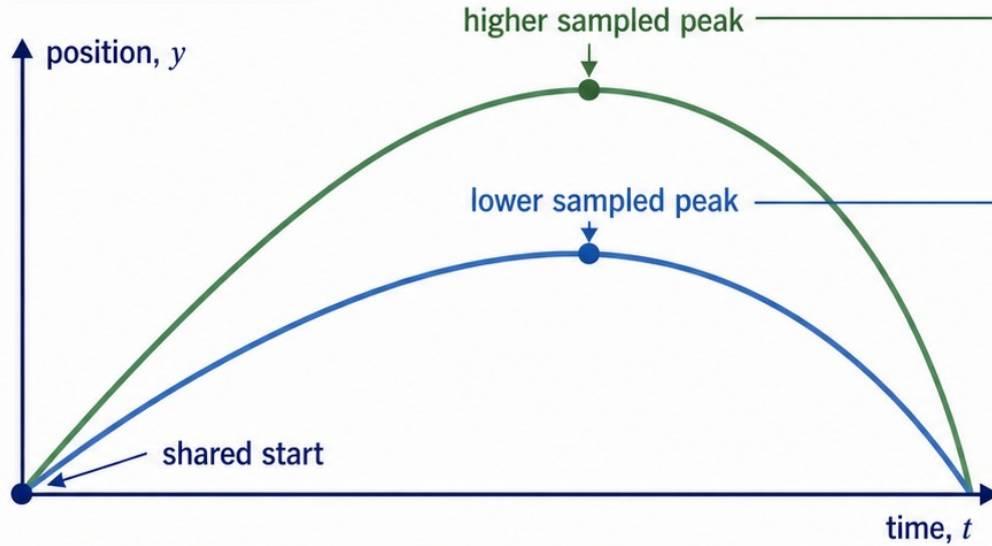
The first stored value should remain 0 m.

Change one input, predict one consequence



Read the plot as evidence, not decoration

CONCEPTUAL ONLY



conceptual trend sketch — not numerical data



Both cases begin at 0 m.



The lower launch speed produces a lower sampled peak.



Labels and units make the output interpretable.



The claim is valid only under the stated model assumptions.

Exact values come from the MATLAB output.

Diagnose the defect before editing the code

The code running is not a correctness test.



Syntax defect

Stops MATLAB reading the intended structure

```
for k = 1:numel(t_s)
    ...
% missing end
```

→ MATLAB cannot determine the loop boundary.



Inspect: error location



Indexing or operator defect

Runs, but stores or calculates the wrong thing

```
y_m(1) = ...
```

→ Overwrites one storage slot on every loop pass.



Inspect: pseudocode + output

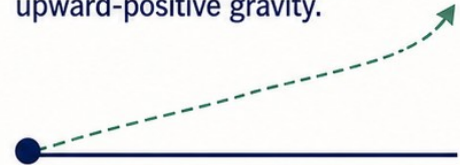


Physical or logical defect

Runs, but contradicts the model

```
y_m(k) = y0_m + v0_mps*t_s(k)
        + 0.5*g_mps2*t_s(k)^2
```

→ Accelerates upward under upward-positive gravity.



Inspect: physics

stops? → **syntax**



storage wrong? → **index/operator**



physics wrong? → **sign/model**

One check can pass while the model is still wrong

Known initial value

$$y(0) = 0 \text{ m}$$

```
assert(abs(y_m(1)-y0_m) < 1e-12)
```



PASS at $t = 0$

Checks the first stored value.

A wrong sign can hide at $t = 0$

$$y_wrong(k) = y0_m + v0_mps*t_s(k) + 0.5*g_mps2*t_s(k)^2$$



wrong: accelerates upward



FAIL physical direction



Numerical check: does the stored value match?



Physical check: does the trajectory match the model?

check the number and the meaning

Exit ticket — explain the loop to a future you



ORDER

Put input, calculation, output, and check in a sensible order.



TRACE

For `sample_index = 2`, what does `t_s(sample_index)` select?



DIAGNOSE

Give one syntax, indexing/operator, or physical/logical defect and its evidence.



VALIDATE

State one validation and one physical conclusion from the graph.

model → algorithm → code → check → interpretation



Next: the same recipe organises a 2×2 circuit model